

Building the AI Engine

Version 1, 2026-06-09

Introduction & Overview

The Shadow AI Crisis

The CIO at your enterprise just called an emergency meeting. She projects a slide showing two deeply concerning charts.

"We have a \$1.5 million 'Shadow AI' problem," she begins, pointing to the first chart. "Developers across five different departments are expensing unmanaged, external LLM API keys. We have zero visibility into what data is leaving our network or how many tokens we are consuming."

She points to the second chart. "At the same time, we've invested over \$500,000 in internal GPU infrastructure. Yet, our data scientists are waiting weeks for capacity because resources are hoarded by single teams or sitting idle."

The business needs to regain control. By centralizing AI operations, enterprises have realized over \$5 million in cost avoidance, achieved a 40%+ reduction in GPU hours, and delivered 2x faster model responses.

Your mandate is clear: Transform this fragmented chaos into a secure, governed, and highly scalable **Enterprise AI Factory**.

Example 1. Arcade: Stepping into the AI Engine Room

(Visual: A split screen showing chaotic, fragmented infrastructure on the left, transitioning to a streamlined, automated, and secure factory pipeline on the right.)

Narrator: "Welcome to 'A Practical Approach to OpenShift AI 3.4.' In this course, we will build a centralized, governed AI engine room from the ground up. You will learn how to enforce token limits to control API costs, securely deploy massive models across distributed hardware, and empower your data scientists and developers with self-service, compliant toolchains. Let's step out of the shadows and get to work."

Course Prerequisites & Environment

This is a practical, hands-on series. Rather than just showing you the "happy path," we will walk through the exact steps to build this architecture while navigating the configurations and capabilities you will face in production.

To get the most out of this course, you should understand:

- Red Hat OpenShift administration basics.
- Generic GPU enablement and Kubernetes resource requests.
- Basic API and networking principles.

Product Overview by Persona

OpenShift AI 3.4 brings massive updates that directly solve our CIO's crisis, targeting three core roles within the enterprise.

The Platform Engineer (Your Role)

You are responsible for securing the infrastructure, optimizing hardware utilization, and providing visibility into resource consumption.

- **Models-as-a-Service (MaaS) & Observability:** Replaces fragmented API keys with a flexible, internal subscription model. Define subscriptions to grant specific groups access to models with strict token quotas, and use the unified dashboard to generate chargeback reports for finance.
- **Distributed Inference (LLM-d):** Overcomes the lack of single-node GPU capacity for massive models by providing a Kubernetes-native framework to serve large language models at scale across multiple nodes.
- **Kueue for Hardware Sharing:** Maximizes your GPU ROI by intelligently managing quotas and prioritizing job scheduling, stopping the "GPU turf wars."
- **MCP Catalog:** A centralized UI for discovering and securely deploying Model Context Protocol (MCP) servers to prevent unvetted agentic integrations.

The Data Scientist

Focused on discovering the right models, ensuring their accuracy, and optimizing Retrieval-Augmented Generation (RAG) pipelines.

- **Model Catalog & "Tool Calling" Filter:** A curated library of generative AI models. The new "Tool calling" filter instantly identifies models validated for agentic workflows.
- **EvalHub:** An evaluation orchestration service that tests LLMs against standardized benchmarks to guarantee safety and compliance.
- **AutoRAG (Technology Preview):** Automatically tests combinations of chunking, embedding, and retrieval settings to rank the best configurations for your RAG pipelines.

The Application Developer

Needs frictionless, self-service access to models and tools to build intelligent applications quickly.

- **Gen AI Playground:** An interactive chat environment to rapidly prototype foundation models and test external tools before writing any application code.
- **Self-Service API Keys:** Developers can instantly generate short-lived temporary API keys for prototyping, or permanent keys tied to their MaaS subscriptions, removing IT bottlenecks.

Feature-to-Role Alignment Matrix (Your Runbook)

Business Challenge	Persona	OpenShift AI 3.4 Feature	UI Action / Resolution
Spiraling API Costs & Lack of Showback	Platform Engineer	Models-as-a-Service (MaaS) & Observability	Navigate to Settings → Subscriptions to set token limits, and Observe & monitor → Dashboard to export usage data.
Hoarded GPUs & Model Capacity Limits	Platform Engineer	Kueue & Distributed Inference (llm-d)	Deploy models using the Distributed inference with llm-d runtime and apply Kueue labels to share resources.
Unvetted Shadow Agentic AI	Data Scientist	Model Catalog & EvalHub	Filter the Catalog using the Tool calling checkbox, then orchestrate benchmark tests via EvalHub.
Friction in API Credential Management	Application Developer	Gen AI Playground & API Keys	Access Gen AI studio → API keys to generate scoped tokens securely.

Diagnostic Gateway: Knowledge Check

Before we move into the lab environment to solve the CIO's challenges, let's verify your core understanding.

Test Your Knowledge

Scenario: Your Finance department is demanding an LLM cost showback report by Friday to identify which development team is spending the most on AI inference.

Question: Which OpenShift AI 3.4 feature allows you to natively track token consumption and generate showback reports for specific user groups?

- A) EvalHub
- B) Models-as-a-Service (MaaS)
- C) Model Registry
- D) AutoRAG

(Answer: B - Models-as-a-Service allows you to centrally manage subscriptions, token rate limits, and view unified observability dashboards.)

What's Next: Your IT Support Tickets

In the upcoming interactive lab modules, you will step into the AI Engine Room to resolve the following IT tickets:

- **Ticket 1:** Configure a MaaS subscription and authorization policy to securely govern developer API access and enforce strict token quotas.

- **Ticket 2:** Discover and deploy a massive agentic model using the Distributed Inference (**llm-d**) runtime to bypass single-node GPU limits.
- **Ticket 3:** Interact with external MCP servers through the Gen AI Playground to test complex tool execution safely.

Ready to start building? Proceed to Module 2.

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Interactive Quiz

```
<iframe type="text/javascript" src='_attachments/s1-quiz.html' style="width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
```

```
:docname: section1
:page-module: chapter1
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: Platform Setup
:description: Theoretical architecture and component mapping for OpenShift AI 3.4
setup.
[#chapter1:section1]
=== Runbook: Platform Setup
```

If you passed the Diagnostic Gateway, use this page as your quick-reference job aid. If you are building your foundational skills, read this carefully before proceeding to the interactive labs.

[discrete#chapter1:section1::_understanding_the_architecture_layers]

==== Understanding the Architecture Layers

Setting up OpenShift AI is not a single-click installation; it is a layered architecture. Platform Engineers must understand the distinction between the management plane and the activation plane.

* **The Operator (Management Plane):** Installing the Red Hat OpenShift AI Operator from OperatorHub simply lays down the management plane. It does *not* actually install the AI components (like Jupyter, MLflow, or KServe).

* **DataScienceClusterInitialization (DSCI - The Foundation):** The DSCI custom resource handles global, cluster-wide configurations that must exist before components spin up. This includes injecting cluster-wide trusted CA certificate bundles and enabling the centralized OpenTelemetry/Prometheus observability stack.

* **DataScienceCluster (DSC - The Activation Layer):** The DSC custom resource defines exactly which components are instantiated in the cluster by setting their `'managementState'` to `'Managed'` or `'Removed'`.

Note for 3.4: The MLflow Operator is now a natively managed component directly within the DSC, removing the need for legacy feature flags.

[discrete#chapter1:section1::_the_paradigm_shift_the_kubernetes_gateway_api]

==== The Paradigm Shift: The Kubernetes Gateway API

In previous versions of OpenShift AI, configuring KServe required OpenShift Service Mesh and OpenShift Serverless (Knative).

****In OpenShift AI 3.4, the KServe Serverless deployment mode is officially deprecated.****

Instead, Platform Engineers must use ****KServe RawDeployment mode****, which leverages the Kubernetes Gateway API for path-based routing. This modern architecture replaces individual OpenShift Routes and sidecar-heavy Service Mesh architectures with a single Gateway entry point (e.g., 'openshift-ai-inference' Gateway). Advanced routing, token authentication, and distributed serving ('llm-d') are now handled natively via the Gateway API paired with Red Hat Connectivity Link (Kuardant).

[discrete#chapter1:section1::_selecting_the_right_3_4_serving_runtime]

==== Selecting the Right 3.4 Serving Runtime

OpenShift AI 3.4 provides highly specialized runtimes out-of-the-box. Selecting the wrong runtime will cause immediate Out-Of-Memory (OOM) errors or massive latency.

* ****vLLM ServingRuntime:**** The standard for serving Single-Node Large Language Models on NVIDIA/AMD GPUs. It uses advanced techniques like PagedAttention and continuous batching.

* ****Distributed Inference ('llm-d'):**** Used when an LLM is too large to fit into the memory of a single GPU. It spans model weights across multiple nodes/GPUs using Ray and the Leader Worker Set operator.

* ****MLServer ServingRuntime:**** Now GA in 3.4, this is the designated runtime for classical, structured-data machine learning models built with Scikit-Learn, XGBoost, and LightGBM.

[discrete#chapter1:section1::_day_2_diagnostics_the_anti_pattern]

==== Day-2 Diagnostics: The Anti-Pattern

Can you spot the architectural flaw?

****The Scenario:**** A junior admin deployed the OpenShift AI 3.4 Operator and activated KServe in the DSC. A Data Scientist deploys a model, but external applications cannot reach the endpoint. The junior admin decides to manually write a Kubernetes 'Route' YAML file exposing port 8080 of the model's pod to the public internet.

****Why this is an Anti-Pattern:**** By bypassing the Gateway API and creating a manual route, the admin has completely broken the platform's security and observability. The Models-as-a-Service (MaaS) token authentication, rate limiting, and Prometheus metrics scraping rely entirely on traffic passing through the centralized 'openshift-ai-inference' Gateway. In 3.4, external ingress should *only* be configured by checking the "Make deployed models available through an external route" box in the UI, which registers the route securely with the Gateway.

[discrete#chapter1:section1::_configuration_cheat_sheet_openshift_ai_3_4]

==== Configuration Cheat Sheet: OpenShift AI 3.4

As a Platform Engineer, you will be expected to know these specific configurations to ensure the platform operates smoothly and upgrades safely:

* ****Operator Upgrades (OLM):**** When installing the operator, always select the

****Manual**** update approval strategy. OpenShift AI requires sequential updates. "Manual" ensures you don't accidentally skip a required intermediate minor version during cluster maintenance.

*** **Monitoring User Workloads:**** Activating the observability stack in the DSCI monitors the **platform**. To ensure Prometheus automatically scrapes metrics from a data scientist's custom model pod, you must apply the ``monitoring.opendatahub.io/scrape=true`` label to their workload.

*** **Prerequisites for `llm-d`:**** If you plan to use Distributed Inference (``llm-d``), the OpenShift AI operator alone is not enough. You must pre-install the ****cert-manager Operator****, the ****Red Hat Connectivity Link Operator**** (for gateway routing), and the ****Leader Worker Set Operator**** (to manage the multi-node pod topology).

<<<

Ready to build it? Proceed to the Interactive Labs.

```
:!navtitle:
:!description:

:docname: section2
:page-module: chapter1
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: Building the Engine Room
:description: Interactive UI click-paths for installing OpenShift AI 3.4.
[#chapter1:section2]
=== Labs: Building the Engine Room
```

It is time to step into the Engine Room. In these interactive labs, you will execute the "Happy Path" to install the Operator, activate the centralized observability stack, and deploy a smoke-test model using the new Gateway API routing.

```
[discrete#chapter1:section2::_lab_1_operator_installation_core_setup]
```

```
==== Lab 1: Operator Installation & Core Setup
```

****Goal:**** Install the OpenShift AI Operator and activate the core components using the ``DataScienceCluster`` resource.

.Arcade: Install and Activate

```
====
```

```
// Antora iframe embed placeholder for Arcade 1
```

```
// +++++
```

```
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
```

```
// +++++
```

****Guided Steps:****

1. In the OpenShift web console, log in as a cluster administrator.
2. Navigate to ****Operators -> OperatorHub**** and install ****Red Hat OpenShift AI****.
3. Navigate to ****Operators -> Installed Operators**** and click the Operator.

4. Click the **Data Science Cluster** tab, then click **Create DataScienceCluster**.
5. Select **YAML view**. Change the `managementState` from `Removed` to `Managed` for the following core components: `kserve`, `kueue`, `mlflowoperator`, and `modelregistry`.
6. Click **Create**.

====

[discrete#chapter1:section2::_lab_2_configuring_the_serving_environment_observability]

==== Lab 2: Configuring the Serving Environment (Observability)

Goal: Activate the centralized observability stack to monitor AI health and user model metrics natively.

.Arcade: Enable Observability

====

// Antora iframe embed placeholder for Arcade 2

// +++++

// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>

// +++++

Guided Steps:

1. Navigate to **Operators -> Installed Operators -> Red Hat OpenShift AI**.
2. Click the **DSC Initialization** tab and open the `default-dsci` instance.
3. Select the **YAML** tab. Under `spec.monitoring`, set `managementState: Managed` to deploy the Prometheus and OpenTelemetry stack. Click **Save**.
4. To enable the Dashboard UI, navigate to **Home -> API Explorer** and search for `OdhDashboardConfig`.
5. Open the `odh-dashboard-config` instance YAML. Under `spec.dashboardConfig`, add `observabilityDashboard: true` and click **Save**.

====

[discrete#chapter1:section2::_lab_3_the_smoke_test]

==== Lab 3: The Smoke Test

Goal: Deploy a test model using the UI to verify KServe RawDeployment and Gateway API routing are operational.

.Arcade: Deploy Smoke Test

====

// Antora iframe embed placeholder for Arcade 3

// +++++

// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>

// +++++

Guided Steps:

1. Open the **Red Hat OpenShift AI** dashboard via the Application Launcher.
2. Navigate to **Data Science Projects** and create a project named `smoke-test-project`.
3. In the **Deployments** tab, click **Deploy model**.
4. Fill out the wizard using the **MLServer ServingRuntime** and the **Scikit-learn** framework.
5. **Critical Step:** Under the Model route section, check the box for **"Make deployed**

models available through an external route" to trigger the Gateway API ingress.

6. Click **Deploy** and wait for the status checkmark to turn green.

====

Module Complete! You have successfully laid the foundation. You are now ready to govern this platform using Models-as-a-Service in Module 3.

:!navtitle:

:!description:

:docname: index

:page-module: chapter2

:page-relative-src-path: index.adoc

:page-origin-url: <https://github.com/RedHatQuickCourses/rhoai34-practical>

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

:page-origin-refhash: (worktree)

[#chapter2:index]

== Interactive Quiz #2

++++

```
<iframe type="text/javascript" src='_attachments/s2-quiz.html' style="width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
```

++++

:docname: section1

:page-module: chapter2

:page-relative-src-path: section1.adoc

:page-origin-url: <https://github.com/RedHatQuickCourses/rhoai34-practical>

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

:page-origin-refhash: (worktree)

:navtitle: Runbook: MaaS & Observability

:description: Theoretical architecture, cheat sheet, and Anti-Pattern for MaaS.

[#chapter2:section1]

== Runbook: MaaS & Observability

If you passed the Diagnostic Gateway, use this page as your quick-reference job aid.

If you are building your foundational skills, read this carefully before proceeding to the interactive labs.

[discrete#chapter2:section1::_the_centralized_gateway_vs_direct_static_api_keys]

==== The Centralized Gateway vs. Direct Static API Keys

In traditional "Shadow AI" environments, developers bypass IT by directly requesting and embedding vendor API keys (like OpenAI or AWS Bedrock) into their source code.

This is a massive vulnerability: if a key leaks, there is no way to instantly restrict access, and the enterprise has zero visibility into which team is driving up the cloud bill.

Models-as-a-Service (MaaS) solves this using the MaaS API Gateway (backed by Kuadrant and Red Hat Connectivity Link). Instead of handing out the root provider keys, the Platform Engineer stores the root provider key in a secure Kubernetes Secret and registers an `ExternalModel` resource.

The MaaS Gateway acts as a proxy using a **Two-Tier Authentication** pattern:

1. Users authenticate to the Gateway using scoped, self-service MaaS API Keys.
2. The Gateway tracks their token consumption against their assigned quota, applies rate limits, and then securely injects the root provider key into the outbound request `[Source: Red Hat OpenShift AI Self-Managed-3.4-Govern LLM access with Models-as-a-Service - Two-tier authentication]`.

[discrete#chapter2:section1::_the_architecture_of_access_groups_subscriptions_and_keys]

==== The Architecture of Access: Groups, Subscriptions, and Keys

Access in MaaS 3.4 is decoupled into distinct layers to provide maximum GitOps flexibility:

* **OpenShift User Groups:** Defines the user's identity (e.g., `'data-scientists'`, `'ml-engineers'`).

* **MaaS Subscriptions (`'MaaSSubscription'`):** Maps a User Group to specific models and enforces strict token quotas (e.g., 500k tokens per hour). If a user belongs to multiple groups, the Subscription with the highest Priority dictates their default limits `[Source: Red Hat OpenShift AI Self-Managed-3.4-Govern LLM access with Models-as-a-Service - Priority levels]`.

* **Authorization Policies (`'MaaSAuthPolicy'`):** Grants the User Group the actual network permission to pass through the API Gateway `[Source: Red Hat OpenShift AI Self-Managed-3.4-Govern LLM access with Models-as-a-Service - Models-as-a-Service authorization policies]`.

* **API Keys:** Developers generate their own self-service keys scoped to their Subscription `[Source: Red Hat OpenShift AI Self-Managed-3.4-Govern LLM access with Models-as-a-Service - Self-service API key management]`.

[discrete#chapter2:section1::_configuration_cheat_sheet_granting_a_new_team_access]

==== Configuration Cheat Sheet: Granting a New Team Access

To provision access for a new team, a Platform Engineer completes these exact steps:

1. **Publish the Model:** Create a `'MaaSModelRef'` (for internal models) or an `'ExternalModel'` (for external APIs like OpenAI) to register the endpoint.
2. **Create the Subscription:** Navigate to **Settings -> Subscriptions**. Assign the OpenShift User Group, select the model, and define the Token Rate Limit (e.g., 1,000,000 / 1h).
3. **Create the Authorization Policy:** Check the "Create matching authorization policy" box during Subscription creation, or manually create a `'MaaSAuthPolicy'` matching the Group to the Model.
4. **Inform the Developer:** Instruct developers to navigate to **Gen AI studio -> API keys** to generate their scoped, self-service tokens.

[discrete#chapter2:section1::_day_2_diagnostics_the_anti_pattern]

==== Day-2 Diagnostics: The Anti-Pattern

Can you spot the architectural flaw?

****The Scenario: The Shadow AI Key Leak****

Your company decides to experiment with OpenAI's GPT-4o. To move fast, the Lead Platform Engineer creates a single enterprise OpenAI account, generates a root API key, and emails it to the 50 developers on the Analytics team. The developers embed this key directly into their application code (`Authorization: Bearer sk-openai-...`).

****Why this is an Anti-Pattern:****

* ****No Showback:**** Finance receives a single massive bill from OpenAI at the end of the month, with zero ability to determine which developer or application is responsible.

* ****No Quota Enforcement:**** A runaway loop in a single developer's code can exhaust the entire company's provider-level token limit, taking down production applications for everyone.

* ****Security Nightmare:**** If a developer accidentally commits the root key to a public GitHub repo, the Platform Engineer has to revoke the key at the provider level, instantly breaking all 50 applications simultaneously.

****The Solution:****

The Platform Engineer should use the OpenShift AI 3.4 'ExternalModel' custom resource. The root OpenAI API key is stored securely in a Kubernetes Secret inside the cluster. Developers never see it. Instead, developers generate their own temporary or permanent MaaS API keys via the OpenShift AI dashboard and send requests to the centralized OpenShift AI Gateway.

The Gateway enforces their specific Subscription token limit (preventing runaway code), logs their exact usage for Finance's showback reports in the Observability Dashboard, and safely proxies the request to OpenAI. If a developer's MaaS key leaks, the Admin can simply click "Revoke" on that specific key without affecting the other 49 developers.

<<<

Ready to build it? Proceed to the Interactive Labs.

:!navtitle:

:!description:

:docname: section2

:page-module: chapter2

:page-relative-src-path: section2.adoc

:page-origin-url: <https://github.com/RedHatQuickCourses/rhoai34-practical>

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

:page-origin-refhash: (worktree)

:navtitle: Labs: MaaS & Observability

:description: Interactive UI click-paths for configuring OpenShift AI 3.4 MaaS.

[#chapter2:section2]

=== Labs: MaaS & Observability

In these interactive labs, you will execute the "Happy Path" to securely proxy an external LLM, govern user access with strict quotas, and view token showback reports for finance.

[discrete#chapter2:section2::_lab_1_connecting_an_external_provider]

==== Lab 1: Connecting an External Provider

****Goal:**** Securely register an external model (GPT-4o) using the OpenShift console so developers don't need the root API key.

.Arcade: Configure External Egress

====

// Antora iframe embed placeholder for Arcade 1

// +++++

// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>

// +++++

****Guided Steps:****

1. Log in to the OpenShift Console as a cluster administrator.
2. Create the root credential: Open the terminal and run: ``oc create secret generic openai-secret --from-literal=api-key=<YOUR_OPENAI_KEY> -n external-models``.
3. Navigate to ****Administration -> CustomResourceDefinitions****.
4. Search for ``ExternalModel`` and click the resource name.
5. Click the ****Instances**** tab, select the ``external-models`` project, and click ****Create ExternalModel****.
6. Configure the YAML: Set ``spec.provider: openai``, ``spec.endpoint: api.openai.com``, ``spec.targetModel: gpt-4o``, and ``spec.credentialRef.name: openai-secret``.
7. Click ****Create****. The model is now registered in the MaaS Gateway.

====

[discrete#chapter2:section2::_lab_2_creating_the_subscription_and_auth_policy]

==== Lab 2: Creating the Subscription and Auth Policy

****Goal:**** Assign the "Analytics" team a strict quota for the newly registered GPT-4o model.

.Arcade: Define Subscription Quotas

====

// Antora iframe embed placeholder for Arcade 2

// +++++

// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>

// +++++

****Guided Steps:****

1. Open the OpenShift AI Dashboard and navigate to ****Settings -> Subscriptions****.
2. Click ****Create subscription****. Name it ``analytics-gpt-sub``.
3. Set the Priority level to ``10``.
4. Under Groups, select the OpenShift group ``analytics-team``.
5. Click ****Add models****, select your new ``gpt-4o`` external model, and click ****Add models****.
6. Click ****Add token limit****. Set the limit to ``500000`` tokens per ``1 hour``. Click ****Save****.
7. Check the box for ****"Create a matching authorization policy"**** to simultaneously

grant API gateway access.

8. Click **Create subscription**.

====

[discrete#chapter2:section2::_lab_3_generating_keys_monitoring_showback]

==== Lab 3: Generating Keys & Monitoring Showback

Goal: Step into the shoes of a Developer generating a key, then pivot back to the Admin to view the financial showback report.

.Arcade: Developer Self-Service and Showback

====

// Antora iframe embed placeholder for Arcade 3

// +++++

// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>

// +++++

Guided Steps:

1. *(As the Developer):* Navigate to **Gen AI studio -> API keys**.

2. Click **Create API key**. Name it 'dev-testing-key'.

3. Select the 'analytics-gpt-sub' subscription, set expiration to 30 days, and click **Create**. Copy the 'sk-oai-...' prefix key before closing the dialog.

4. *(As the Admin):* Navigate to **Observe & monitor -> Dashboard**.

5. Click the **Usage** tab to view the MaaS observability metrics.

6. Review the **Token Consumption by User** table to see exactly how many tokens the developer consumed against the 'gpt-4o' endpoint.

7. Click **Export as CSV** to download the billing showback report for Finance.

====

Module Complete! You have successfully eliminated shadow AI by providing developers self-service access while maintaining strict financial and security controls.

:!navtitle:

:!description:

:docname: index

:page-module: chapter3

:page-relative-src-path: index.adoc

:page-origin-url: <https://github.com/RedHatQuickCourses/rhoai34-practical>

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

:page-origin-refhash: (worktree)

[#chapter3:index]

== Interactive Quiz #3

++++

<iframe type="text/javascript" src='_attachments/s3-quiz.html' style="width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>

++++

```
:docname: section1
:page-module: chapter3
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: Distributed Inference & Kueue
:description: Theoretical architecture and Anti-Pattern for Kueue and llm-d.
[#chapter3:section1]
=== Runbook: Distributed Inference & Kueue
```

If you passed the Diagnostic Gateway, use this page as your quick-reference job aid. If you are building your foundational skills, read this carefully before proceeding to the interactive labs.

```
[discrete#chapter3:section1::_why_massive_llms_require_distributed_inference]
```

```
==== Why Massive LLMs Require Distributed Inference
```

A common misconception is that standard Kubernetes autoscaling or adding more Pod replicas will solve Out of Memory (OOM) errors for large models. It will not.

If a data science team attempts to load a 70-Billion parameter model at 16-bit precision, the model weights alone require approximately 140GB of VRAM. If your cluster only has standard GPUs with 24GB to 80GB of VRAM, the model cannot physically fit on a single device.

Distributed Inference with 'llm-d' solves this by applying Tensor Parallelism and Pipeline Parallelism. Instead of duplicating the entire model per pod, 'llm-d' leverages the Red Hat **Leader Worker Set (LWS) Operator** (which officially replaces Ray in 3.4 for vLLM workloads) to slice the model's layers and distribute them across multiple GPUs and nodes.

```
[discrete#chapter3:section1::_kueue_the_gpu_traffic_cop]
```

```
==== Kueue: The GPU Traffic Cop
```

Without Kueue, if ten data scientists simultaneously request a 4-GPU distributed inference deployment on an 8-GPU cluster, Kubernetes will attempt to schedule all of them, resulting in 'InsufficientQuota' crashes or deadlocks.

Kueue acts as a centralized traffic cop. It uses a **ClusterQueue** to define the global limits of your hardware (e.g., maximum 8 GPUs total) and **LocalQueues** in user namespaces that point to the ClusterQueue. If resources are full, Kueue gracefully holds the new workloads in a "Pending" state in the LocalQueue until the required GPUs are freed up.

```
[discrete#chapter3:section1::_configuration_cheat_sheet_preparing_for_distributed_models]
```

```
==== Configuration Cheat Sheet: Preparing for Distributed Models
```

1. **Install Prerequisites:** Install the independent Red Hat build of Kueue Operator, Node Feature Discovery, NVIDIA GPU Operator, and the Leader Worker Set Operator.

2. ****Enable Kueue:**** In the DataScienceCluster CR, set `'spec.components.kueue.managementState'` to `'Unmanaged'` (as the embedded Kueue component is deprecated in 3.4).
3. ****Define Quotas:**** Create a `'ResourceFlavor'` (defining the node types) and a `'ClusterQueue'` (defining the total GPU limits).
4. ****Create the Local Queue:**** In the target project namespace, create a `'LocalQueue'` that references the `'ClusterQueue'`.
5. ****Create a Hardware Profile:**** In the OpenShift AI Dashboard (****Settings -> Hardware profiles****), create a profile with the required GPUs. Under "Workload allocation strategy", select ****Local queue**** and input your LocalQueue name.
6. ****Deploy:**** Data scientists select `'Distributed inference with llm-d'` as the serving runtime and apply the new Hardware Profile during deployment.

[discrete#chapter3:section1::_day_2_diagnostics_the_anti_pattern]

==== Day-2 Diagnostics: The Anti-Pattern

Can you spot the architectural flaw?

****The Scenario: The "More Replicas" Fallacy****

A platform engineer is asked to deploy the `'llama-3.1-70B'` model. They spin up a standard vLLM Serving Runtime on a cluster where nodes only have 24GB GPUs. The deployment instantly crashes with an Out of Memory (OOM) error. Thinking this is a throughput capacity issue, the engineer edits the InferenceService and increases `'spec.replicas'` from 1 to 4, hoping the load will spread out and the model will load. All 4 replicas immediately crash.

****Why this is an Anti-Pattern:****

This is a fundamental misunderstanding of VRAM memory footprint vs. HTTP request throughput. Requesting more replicas of a standard vLLM runtime does not split the model; it instructs Kubernetes to spin up 4 identical Pods, each attempting to cram the entire 140GB model into a single 24GB GPU. It just copies the failure four times.

****The Solution:****

The engineer must use the ****Distributed Inference with `'llm-d'`**** runtime. By configuring `'workerSpec.tensorParallelSize'` (e.g., setting it to 8), the `'llm-d'` runtime pairs with the Leader Worker Set operator to actually **shard** the model weights across 8 GPUs of 24GB each. This ensures the aggregate memory can easily accommodate the massive LLM.

<<<

Ready to build it? Proceed to the Interactive Labs.

:!navtitle:

:!description:

:docname: section2

:page-module: chapter3

:page-relative-src-path: section2.adoc

:page-origin-url: <https://github.com/RedHatQuickCourses/rhoai34-practical>

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

```
:page-origin-refhash: (worktree)
:navtitle: Labs: Distributed Inference
:description: Interactive UI click-paths for configuring OpenShift AI 3.4 Kueue and
llm-d.
[#chapter3:section2]
=== Labs: Distributed Inference
```

In these interactive labs, you will execute the "Happy Path" to link Kueue to a Hardware Profile and deploy a massive model across multiple GPUs.

```
[discrete#chapter3:section2::_lab_1_stopping_the_gpu_turf_war_kueue]
```

```
==== Lab 1: Stopping the GPU Turf War (Kueue)
```

****Goal:**** Ensure Kueue is enabled so workloads pend gracefully instead of crashing the cluster.

.Arcade: Enable Kueue and LocalQueues

```
=====
```

```
// Antora iframe embed placeholder for Arcade 1
```

```
// +++++
```

```
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
```

```
// +++++
```

****Guided Steps:****

1. Log in to the OpenShift AI dashboard as an administrator.
2. Navigate to ****Settings -> Cluster settings****.
3. Locate the Kueue section and ensure the UI integration is active.
4. Navigate to your OpenShift Console terminal and ensure your data science project is labeled for Kueue enforcement: ``oc label namespace <your-project> kueue.openshift.io/managed=true``.
5. Apply a basic ``LocalQueue`` YAML to your namespace named ``heavy-training-queue`` that points to your cluster's default ``ClusterQueue``.

```
=====
```

```
[discrete#chapter3:section2::_lab_2_creating_a_hardware_profile]
```

```
==== Lab 2: Creating a Hardware Profile
```

****Goal:**** Create a UI-selectable profile that automatically links distributed workloads to the Kueue system.

.Arcade: Hardware Profiles

```
=====
```

```
// Antora iframe embed placeholder for Arcade 2
```

```
// +++++
```

```
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
```

```
// +++++
```

****Guided Steps:****

1. In the OpenShift AI dashboard, navigate to ****Settings -> Hardware profiles****.
2. Click ****Create hardware profile****. Name it ``Distributed GPU Node``.
3. Click ****Add resource****. Set Resource identifier to ``nvidia.com/gpu`` and ensure Requests/Limits are set appropriately for a multi-GPU slice.
4. Scroll down to the ****Resource allocation**** section.

5. Under **Workload allocation strategy**, select **Local queue**.
6. In the Local queue field, enter 'heavy-training-queue'.
7. Click **Create hardware profile**.

====

[discrete#chapter3:section2::_lab_3_deploying_a_distributed_model]

==== Lab 3: Deploying a Distributed Model

Goal: Deploy a massive model using llm-d, leveraging the queue and hardware profile.

.Arcade: Deploying llm-d

====

// Antora iframe embed placeholder for Arcade 3

// +++++

// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>

// +++++

Guided Steps:

1. In the OpenShift AI dashboard, navigate to **Data Science Projects** and open your project.
2. Click the **Deployments** tab.
3. In the Model serving platform tile, click **Deploy model**.
4. Name the deployment 'massive-70b-model'.
5. Under **Hardware profile**, select your newly created 'Distributed GPU Node'.
6. Under **Serving runtime**, explicitly select 'Distributed inference with llm-d'.
7. Provide your S3-compatible storage connection containing the model weights.
8. Click **Deploy**. Because you assigned the hardware profile tied to Kueue, the underlying service is automatically labeled with 'kueue.x-k8s.io/queue-name: heavy-training-queue' and will pend gracefully if GPUs are currently in use.

====

Module Complete! You have successfully eliminated GPU resource conflicts and scaled inference beyond the limits of a single node.

:!navtitle:

:!description:

:docname: index

:page-module: chapter4

:page-relative-src-path: index.adoc

:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical

:page-origin-start-path:

:page-origin-refname: main

:page-origin-reftype: branch

:page-origin-refhash: (worktree)

[#chapter4:index]

== Interactive Quiz #4

++++

<iframe type="text/javascript" src='_attachments/s4-quiz.html' style="width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen

```
allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
++++
```

```
:docname: section1
:page-module: chapter4
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: MCP & EvalHub
:description: Theoretical architecture and Anti-Pattern for MCP and EvalHub.
[#chapter4:section1]
=== Runbook: MCP & EvalHub
```

If you passed the Diagnostic Gateway, use this page as your quick-reference job aid. If you are building your foundational skills, read this carefully before proceeding to the interactive labs.

```
[discrete#chapter4:section1::_the_shift_to_agentic_ai_and_the_security_paradigm]
==== The Shift to Agentic AI and the Security Paradigm
```

Historically, AI deployments were simple Chatbots: a user asked a question, and the LLM returned text. With "Agentic AI," the paradigm has flipped. The LLM is now an agent making decisions to actively call external tools, execute code, or query databases to accomplish a goal.

If developers implement this by embedding raw API keys (for AWS, GitHub, databases) into their application code to manually glue the LLM to these tools, they create massive security vulnerabilities.

The **Model Context Protocol (MCP)** solves this by standardizing how models connect to data sources and tools. Instead of custom integration scripts with scattered API keys, the Platform Engineer deploys pre-built, secure MCP Servers (like the Red Hat OpenShift, Ansible, or partner servers) via the **MCP Catalog**. The infrastructure handles the tool execution, and the LLM merely requests the standardized tool format.

```
[discrete#chapter4:section1::_the_value_of_evalhub_securing_ciso_approval]
==== The Value of EvalHub: Securing CISO Approval
```

Deploying an Agentic model without empirical safety data is a guaranteed CISO blocker. You cannot rely on "vibes-based" manual chat testing to determine if an agent will exfiltrate data.

EvalHub provides an enterprise-grade orchestration layer to automate and standardize LLM evaluation. It allows Platform Engineers to submit evaluation jobs against industry-standard benchmarks.

- * For Agentic security, EvalHub integrates the **garak** framework to automatically run security vulnerability scans (like OWASP LLM Top 10 and AVID taxonomy).

- * Crucially, EvalHub seamlessly integrates with **MLflow** (now natively managed in OpenShift AI 3.4), ensuring every evaluation run is tracked, versioned, and auditable

for compliance.

[discrete#chapter4:section1::_configuration_cheat_sheet_secure_agentic_deployment]
==== Configuration Cheat Sheet: Secure Agentic Deployment

1. ****Vetting the Model:**** Go to ****AI hub -> Catalog****, check the ****Tool calling**** filter, and copy the validated CLI deployment arguments (e.g., `--enable-auto-tool-choice`).
2. ****Evaluating the Model:**** Submit an EvalHub job specifying the model endpoint and selecting safety benchmarks (e.g., garak OWASP Top 10). Review the automated audit trail in the MLflow UI.
3. ****Deploying the Tool:**** As an admin, navigate to ****AI hub -> MCP servers (Catalog)****, find the required tool, and deploy the MCP Server directly into the namespace via the `mcp-lifecycle-operator`.
4. ****Testing Safely:**** Instruct developers to open the ****Gen AI Playground****, click the ****MCP**** tab, securely authorize the server with a session token, and test the agent interactively.

[discrete#chapter4:section1::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern

Can you spot the architectural flaw?

****The Scenario: The "Shadow Agent" Exfiltration Risk****

A developer on the logistics team wants to build an AI agent capable of querying real-time shipping events from a secure internal Confluent Kafka stream. To bypass the "slow" IT provisioning process, they embed their personal Confluent Cloud API credentials directly into a LangChain Python script running locally. They expose the agent to the internet to test it from their phone.

****Why this is an Anti-Pattern:****

- * The developer has created a "Shadow Agent." The LLM has unrestricted access to the Kafka stream via the developer's raw API credentials.
- * If the LLM is subjected to a prompt injection attack, a malicious actor could instruct the agent to query the Kafka stream and exfiltrate sensitive shipping manifests to an external URL.
- * Because the integration is hard-coded locally, the enterprise has zero visibility, no audit logs, and no centralized way to revoke the tool access if compromised.

****The Solution:****

In OpenShift AI 3.4, the Platform Engineer eliminates this risk by using the ****MCP Catalog****. Instead of the developer hardcoding keys, the Platform Engineer navigates to the MCP Catalog and deploys the Red Hat technology partner Confluent Cloud MCP server.

The API credentials are securely managed by the cluster via the `mcp-gateway`. The developer simply opens the Gen AI Playground, navigates to the MCP tab, and checks the box for the Confluent tool. They can prototype the agent safely within the governed OpenShift environment without ever touching the raw API keys.

<<<

Ready to build it? Proceed to the Interactive Labs.

```

:!navtitle:
:!description:

:docname: section2
:page-module: chapter4
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: MCP & EvalHub
:description: Interactive UI click-paths for vetting models and deploying MCP servers.
[#chapter4:section2]
=== Labs: MCP & EvalHub

```

In these interactive labs, you will execute the "Happy Path" to find a validated model, deploy a secure integration tool, and test it as a developer.

```

[discrete#chapter4:section2::_lab_1_vetting_an_agentic_model_model_catalog]
==== Lab 1: Vetting an Agentic Model (Model Catalog)
**Goal:** Ensure you select a model empirically validated for agentic workflows,
avoiding models that will hallucinate tool calls.

```

.Arcade: Filtering for Tool Calling

```

====
// Antora iframe embed placeholder for Arcade 1
// +++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
// +++++

```

****Guided Steps:****

1. Log in to the OpenShift AI dashboard.
2. In the left navigation menu, click ****AI hub -> Catalog****.
3. In the left filter pane, locate the Task section and select the ****Tool calling**** checkbox. The catalog will refresh to display only models validated for tool-use compatibility.
4. Click on a validated model card (e.g., `Ministral-3-14B-Instruct-2512`) to open its details page.
5. Scroll to the YAML frontmatter section on the model card and locate the tool-calling metadata.
6. Copy the specific required CLI arguments (e.g., `--enable-auto-tool-choice`, `--tool-call-parser`) and the chat template.
7. Click ****Deploy model****.
8. In the deployment wizard, paste the copied CLI arguments into the ***Additional serving runtime arguments*** text box to ensure the model deploys with its agentic capabilities activated. Click ****Deploy****.

```

====

```

```

[discrete#chapter4:section2::_lab_2_deploying_a_trusted_tool_mcp_catalog]
==== Lab 2: Deploying a Trusted Tool (MCP Catalog)

```

****Goal:**** Deploy a pre-packaged tool from the Catalog so developers can access backend systems without handling raw API credentials.

.Arcade: Deploying an MCP Server

====

```
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++
```

****Guided Steps (Admin):****

1. Ensure the `mcp-lifecycle-operator` is installed on the cluster.
2. In the OpenShift AI dashboard, navigate to ****AI hub -> MCP servers****.
3. Browse the catalog and click on a trusted integration (e.g., the Confluent Cloud or GitHub MCP Server).
4. Review the tool's capabilities and required environment variables (which will be securely stored as secrets).
5. Click ****Deploy**** and select the target developer namespace (`logistics-agent-project`).
6. The `mcp-gateway` now securely hosts this tool endpoint for the namespace.

====

[discrete#chapter4:section2::_lab_3_testing_the_agent_safely_gen_ai_playground]

==== Lab 3: Testing the Agent Safely (Gen AI Playground)

****Goal:**** Act as the developer connecting to the governed tool using the UI, verifying the agentic workflow is operational.

.Arcade: Connecting Tools in the Playground

====

```
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++
```

****Guided Steps (Developer):****

1. In the OpenShift AI dashboard, navigate to ****Gen AI studio -> Playground****.
2. Select your newly deployed tool-capable model from the model dropdown.
3. On the right-hand panel, click the ****MCP**** tab.
4. You will see the MCP Server deployed by the Admin in Lab 2. Click the ****Auth**** icon to authorize your session securely.
5. ***Enable Safety:** In the Playground settings, toggle on ****Prompt Guard**** to ensure your test inputs are scanned for prompt injections.
6. Type a prompt that requires the tool (e.g., ***"Check the Kafka stream for shipping delays"***).
7. Watch the chat interface as the LLM successfully invokes the tool, receives the data, and formulates an answer—all without you writing a single line of integration code.

====

****Course Complete!**** You have successfully built a secure, scalable, and governed AI Engine Room using OpenShift AI 3.4.

```
:!navtitle:
:!description:

:docname: appendix
:page-module: appendix
:page-relative-src-path: appendix.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
[#appendix:appendix]
== Appendix

FIXME: Content for Appendix...

[discrete#appendix:appendix::_resources]
=== Resources

link:{attachmentsdir}/{page-component-name}.pdf[Download Course PDF]
```